ASSIGN 1:

```
import numpy as np

# Define two fuzzy sets
A = np.array([0.2, 0.5, 0.7, 1.0])
B = np.array([0.6, 0.4, 0.8, 0.3])

# Define fuzzy relation matrices
R = np.array([[0.2, 0.7],
              [0.5, 0.9]])

S = np.array([[0.6, 0.3],
              [0.8, 0.4]])


# Function for Union
def fuzzy_union():
    result = np.maximum(A, B)
    print("\nFuzzy Set A:", A)
    print("Fuzzy Set B:", B)
    print("Union (A U B):", result)


# Function for Intersection
def fuzzy_intersection():
    result = np.minimum(A, B)
    print("\nFuzzy Set A:", A)
    print("Fuzzy Set B:", B)
    print("Intersection (A ∩ B):", result)


# Function for Complement
def fuzzy_complement():
    result = 1 - A
    print("\nFuzzy Set A:", A)
    print("Complement of A:", result)
```

```python
# Function for Max-Min Composition
def max_min_composition():
    T = np.zeros((R.shape[0], S.shape[1]))

    for i in range(R.shape[0]):
        for j in range(S.shape[1]):
            T[i, j] = np.max(np.minimum(R[i, :], S[:, j]))

    print("\nRelation Matrix R:\n", R)
    print("\nRelation Matrix S:\n", S)
    print("\nMax-Min Composition (R o S):\n", T)


# Main Menu
while True:

    print("\n========== FUZZY LOGIC OPERATIONS ==========")
    print("1. Fuzzy Union")
    print("2. Fuzzy Intersection")
    print("3. Fuzzy Complement")
    print("4. Max-Min Composition")
    print("5. Display All Operations")
    print("6. Exit")

    choice = input("\nEnter your choice (1-6): ")

    if choice == '1':
        fuzzy_union()

    elif choice == '2':
        fuzzy_intersection()

    elif choice == '3':
        fuzzy_complement()

    elif choice == '4':
        max_min_composition()
```

```
    elif choice == '5':
        fuzzy_union()
        fuzzy_intersection()
        fuzzy_complement()
        max_min_composition()

    elif choice == '6':
        print("\nExiting Fuzzy Logic Program. Thank you!")
        break

    else:
        print("\nInvalid choice! Please enter a number between 1 and 6.")
```

ASSIGN 2:

!pip install scikit-fuzzy

```
!pip install scikit-fuzzy
```

```
import numpy as np
import matplotlib.pyplot as plt
import skfuzzy as fuzz
from skfuzzy import control as ctrl

# 1. Define input and output variables
temperature = ctrl.Antecedent(np.arange(0, 41, 1), 'temperature')
humidity = ctrl.Antecedent(np.arange(0, 101, 1), 'humidity')
cooling = ctrl.Consequent(np.arange(0, 101, 1), 'cooling')

# 2. Define membership functions
temperature['Cold'] = fuzz.trapmf(temperature.universe, [0, 0, 15, 22])
temperature['Comfortable'] = fuzz.trimf(temperature.universe, [18, 24,
30])
temperature['Hot'] = fuzz.trapmf(temperature.universe, [26, 32, 40, 40])
```

```
humidity['Low'] = fuzz.trapmf(humidity.universe, [0, 0, 25, 45])
humidity['Medium'] = fuzz.trimf(humidity.universe, [30, 50, 70])
humidity['High'] = fuzz.trapmf(humidity.universe, [60, 75, 100, 100])

cooling['Low'] = fuzz.trapmf(cooling.universe, [0, 0, 20, 45])
cooling['Medium'] = fuzz.trimf(cooling.universe, [30, 50, 70])
cooling['High'] = fuzz.trapmf(cooling.universe, [55, 75, 100, 100])

# 3. Define fuzzy rules
rules = [
    ctrl.Rule(temperature['Cold'] & humidity['Low'], cooling['Low']),
    ctrl.Rule(temperature['Cold'] & humidity['Medium'], cooling['Low']),
    ctrl.Rule(temperature['Cold'] & humidity['High'], cooling['Low']),

    ctrl.Rule(temperature['Comfortable'] & humidity['Low'],
cooling['Low']),
    ctrl.Rule(temperature['Comfortable'] & humidity['Medium'],
cooling['Medium']),
    ctrl.Rule(temperature['Comfortable'] & humidity['High'],
cooling['Medium']),

    ctrl.Rule(temperature['Hot'] & humidity['Low'], cooling['Medium']),
    ctrl.Rule(temperature['Hot'] & humidity['Medium'], cooling['High']),
    ctrl.Rule(temperature['Hot'] & humidity['High'], cooling['High'])
]

# 4. Create control system
ac_system = ctrl.ControlSystem(rules)

# 5. Calculate cooling output
def calculate_cooling(temp, hum):
    sim = ctrl.ControlSystemSimulation(ac_system)
    sim.input['temperature'] = temp
    sim.input['humidity'] = hum
    sim.compute()

    print("\nFinal Cooling Level:", round(sim.output['cooling'], 2), "%")

    result = sim.output['cooling']
    if result < 40:
```

```
        print("AC Status: LOW COOLING")
    elif result < 70:
        print("AC Status: MEDIUM COOLING")
    else:
        print("AC Status: HIGH COOLING")

    cooling.view(sim=sim)
    plt.title("AC Controller Output")
    plt.show()

# 6. Display membership functions
def plot_membership(variable, title):
    variable.view()
    plt.title(title)
    plt.show()

# 7. Display fuzzy rules
def display_rules():
    print("\n========== FUZZY RULES ==========")
    for i, rule in enumerate(rules, 1):
        print(f"Rule {i}: {rule}")

# 8. Main menu
while True:
    print("\n====== FUZZY AC CONTROLLER ======")
    print("1. Calculate AC Cooling")
    print("2. Temperature Membership Functions")
    print("3. Humidity Membership Functions")
    print("4. Cooling Membership Functions")
    print("5. Display Fuzzy Rules")
    print("6. Exit")

    choice = input("Enter your choice (1-6): ")

    if choice == '1':
        try:
            temp = float(input("Enter Temperature (0-40 °C): "))
            hum = float(input("Enter Humidity (0-100%): "))

            if 0 <= temp <= 40 and 0 <= hum <= 100:
```

```
                calculate_cooling(temp, hum)
            else:
                print("Values are outside the allowed range.")

        except ValueError:
            print("Please enter valid numerical values.")

    elif choice == '2':
        plot_membership(temperature, "Temperature Membership Functions")

    elif choice == '3':
        plot_membership(humidity, "Humidity Membership Functions")

    elif choice == '4':
        plot_membership(cooling, "Cooling Membership Functions")

    elif choice == '5':
        display_rules()

    elif choice == '6':
        print("Exiting AC Controller. Thank you!")
        break

    else:
        print("Invalid choice! Select between 1 and 6.")
```

ASSIGN 3:

```
 !pip install scikit-fuzzy
import numpy as np
import matplotlib.pyplot as plt
import skfuzzy as fuzz
from skfuzzy import control as ctrl
```

```
# ========================================
# 1. DEFINE FUZZY VARIABLES
# ========================================

error = ctrl.Antecedent(
    np.arange(-10, 11, 1), 'error'
)

delta_error = ctrl.Antecedent(
    np.arange(-5, 6, 1), 'delta_error'
)

torque = ctrl.Consequent(
    np.arange(-10, 11, 1), 'torque'
)


# ========================================
# 2. DEFINE MEMBERSHIP FUNCTIONS
# ========================================

# Error Membership Functions
error['negative'] = fuzz.trimf(
    error.universe, [-10, -10, 0]
)

error['zero'] = fuzz.trimf(
    error.universe, [-5, 0, 5]
)

error['positive'] = fuzz.trimf(
    error.universe, [0, 10, 10]
)


# Change in Error Membership Functions
delta_error['decreasing'] = fuzz.trimf(
    delta_error.universe, [-5, -5, 0]
```

```
)

delta_error['stable'] = fuzz.trimf(
    delta_error.universe, [-2, 0, 2]
)

delta_error['increasing'] = fuzz.trimf(
    delta_error.universe, [0, 5, 5]
)


# Torque Membership Functions
torque['negative'] = fuzz.trimf(
    torque.universe, [-10, -10, 0]
)

torque['zero'] = fuzz.trimf(
    torque.universe, [-5, 0, 5]
)

torque['positive'] = fuzz.trimf(
    torque.universe, [0, 10, 10]
)


# ========================================
# 3. DEFINE FUZZY RULES
# ========================================

# Negative error rules
rule1 = ctrl.Rule(
    error['negative'] & delta_error['decreasing'],
    torque['negative']
)

rule2 = ctrl.Rule(
    error['negative'] & delta_error['stable'],
    torque['negative']
)
```

```
rule3 = ctrl.Rule(
    error['negative'] & delta_error['increasing'],
    torque['negative']
)


# Zero error rules
rule4 = ctrl.Rule(
    error['zero'] & delta_error['decreasing'],
    torque['negative']
)

rule5 = ctrl.Rule(
    error['zero'] & delta_error['stable'],
    torque['zero']
)

rule6 = ctrl.Rule(
    error['zero'] & delta_error['increasing'],
    torque['positive']
)


# Positive error rules
rule7 = ctrl.Rule(
    error['positive'] & delta_error['decreasing'],
    torque['positive']
)

rule8 = ctrl.Rule(
    error['positive'] & delta_error['stable'],
    torque['positive']
)

rule9 = ctrl.Rule(
    error['positive'] & delta_error['increasing'],
    torque['positive']
)
```

```
# ========================================
# 4. BUILD FUZZY CONTROL SYSTEM
# ========================================

torque_ctrl = ctrl.ControlSystem([
    rule1, rule2, rule3,
    rule4, rule5, rule6,
    rule7, rule8, rule9
])


# ========================================
# 5. FUNCTION TO CALCULATE TORQUE
# ========================================

def calculate_torque(error_value, delta_value):

    simulation = ctrl.ControlSystemSimulation(
        torque_ctrl
    )

    simulation.input['error'] = error_value
    simulation.input['delta_error'] = delta_value

    simulation.compute()

    return simulation.output['torque'], simulation


# ========================================
# 6. DISPLAY FUZZY RULES
# ========================================

def display_rules():

    print("\n========== FUZZY CONTROL RULES ==========")

    rules = [
        "1. IF Error is Negative AND Change is Decreasing THEN Torque is
Negative",
```

```
        "2. IF Error is Negative AND Change is Stable THEN Torque is
Negative",
        "3. IF Error is Negative AND Change is Increasing THEN Torque is
Negative",
        "4. IF Error is Zero AND Change is Decreasing THEN Torque is
Negative",
        "5. IF Error is Zero AND Change is Stable THEN Torque is Zero",
        "6. IF Error is Zero AND Change is Increasing THEN Torque is
Positive",
        "7. IF Error is Positive AND Change is Decreasing THEN Torque is
Positive",
        "8. IF Error is Positive AND Change is Stable THEN Torque is
Positive",
        "9. IF Error is Positive AND Change is Increasing THEN Torque is
Positive"
    ]

    for rule in rules:
        print(rule)


# ========================================
# 7. PLOT MEMBERSHIP FUNCTIONS
# ========================================

def plot_membership(variable, title):

    variable.view()

    plt.title(title)
    plt.tight_layout()
    plt.show()


# ========================================
# 8. SIMULATE ROBOTIC ARM MOVEMENT
# ========================================

def simulate_robotic_arm():
```

```
    print("\n========== ROBOTIC ARM SIMULATION ==========")

    target_angle = float(
        input("Enter desired arm angle (0-90 degrees): ")
    )

    current_angle = float(
        input("Enter current arm angle (0-90 degrees): ")
    )

    if not (0 <= target_angle <= 90 and
            0 <= current_angle <= 90):

        print("Angles must be between 0 and 90 degrees.")
        return

    velocity = 0.0
    previous_error = None

    dt = 0.1

    print("\nStarting robotic arm simulation...\n")
    print("Step | Arm Angle | Error | Torque")

    for step in range(1, 151):

        # Calculate normalized error
        error_value = np.clip(
            (target_angle - current_angle) / 9,
            -10, 10
        )

        if previous_error is None:
            delta_value = 0.0
        else:
            delta_value = np.clip(
                error_value - previous_error,
                -5, 5
            )
```

```
        previous_error = error_value

        # Calculate fuzzy torque
        torque_value, _ = calculate_torque(
            error_value, delta_value
        )

        # Simplified arm dynamics
        acceleration = (
            3.0 * torque_value - 1.2 * velocity
        )

        velocity = velocity + acceleration * dt

        velocity = np.clip(velocity, -30, 30)

        current_angle = (
            current_angle + velocity * dt
        )

        current_angle = np.clip(
            current_angle, 0, 90
        )

        if step % 10 == 0:
            print(
                f"{step:4d} | {current_angle:9.2f} | "
                f"{target_angle-current_angle:6.2f} | "
                f"{torque_value:6.2f}"
            )

        if (abs(target_angle - current_angle) < 1
                and abs(velocity) < 1):

            print("\nTarget angle reached!")
            break

    print("\nFinal Arm Angle:",
          round(current_angle, 2), "degrees")
```

```
    print("Desired Arm Angle:",
          target_angle, "degrees")

    # Plot arm movement
    plt.figure(figsize=(7, 4))

    plt.bar(
        ["Desired Angle", "Final Angle"],
        [target_angle, current_angle]
    )

    plt.ylabel("Angle (degrees)")
    plt.title("Robotic Arm Position")
    plt.show()


# ========================================
# 9. MAIN MENU
# ========================================

while True:

    print("\n====================================")
    print("   FUZZY ROBOTIC ARM CONTROLLER")
    print("====================================")

    print("1. Calculate Motor Torque")
    print("2. Error Membership Functions")
    print("3. Change in Error Membership Functions")
    print("4. Torque Membership Functions")
    print("5. Display Fuzzy Control Rules")
    print("6. Simulate Robotic Arm Movement")
    print("7. Exit")

    choice = input("\nEnter your choice (1-7): ")

    if choice == "1":

        try:
            e = float(input("Enter Error (-10 to 10): "))
```

```
            de = float(input("Enter Change in Error (-5 to 5): "))

            if not (-10 <= e <= 10):
                print("Error must be between -10 and 10.")

            elif not (-5 <= de <= 5):
                print("Change in Error must be between -5 and 5.")

            else:
                output, simulation = calculate_torque(e, de)

                print("\n========== RESULTS ==========")
                print("Input Error:", e)
                print("Change in Error:", de)
                print("Calculated Motor Torque:",
                      round(output, 2))

                if output > 1:
                    print("Motor Direction: Positive")
                elif output < -1:
                    print("Motor Direction: Negative")
                else:
                    print("Motor Direction: Nearly Zero")

                torque.view(sim=simulation)
                plt.show()

        except ValueError:
            print("Please enter valid numerical values.")

    elif choice == "2":

        plot_membership(
            error,
            "Error Membership Functions"
        )

    elif choice == "3":

        plot_membership(
```

```
            delta_error,
            "Change in Error Membership Functions"
        )

    elif choice == "4":

        plot_membership(
            torque,
            "Motor Torque Membership Functions"
        )

    elif choice == "5":

        display_rules()

    elif choice == "6":

        try:
            simulate_robotic_arm()

        except ValueError:
            print("Please enter valid numerical values.")

    elif choice == "7":

        print("\nExiting Robotic Arm Controller. Thank you!")
        break

    else:

        print("\nInvalid choice! Please select from 1 to 7.")
```

ASSIGN 4:

```
import numpy as np
import random
import matplotlib.pyplot as plt

from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score


# ========================================
# 1. DATASET CREATION
# ========================================

X, y = make_classification(
    n_samples=1000,
    n_features=20,
    n_informative=10,
    random_state=42
)

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)


# ========================================
# 2. HYPERPARAMETER SEARCH SPACE
# ========================================

PARAM_BOUNDS = {
    'n_estimators': (10, 200),
    'max_depth': (2, 20),
    'min_samples_split': (2, 10)
```

```
}


# ========================================
# 3. FITNESS FUNCTION
# ========================================

def evaluate_fitness(individual):

    n_estimators, max_depth, min_samples_split = individual

    model = RandomForestClassifier(
        n_estimators=int(n_estimators),
        max_depth=int(max_depth),
        min_samples_split=int(min_samples_split),
        random_state=42,
        n_jobs=-1
    )

    scores = cross_val_score(
        model,
        X_train,
        y_train,
        cv=3,
        scoring='accuracy'
    )

    return np.mean(scores)


# ========================================
# 4. CREATE RANDOM INDIVIDUAL
# ========================================

def create_individual():

    return [
        random.randint(*PARAM_BOUNDS['n_estimators']),
        random.randint(*PARAM_BOUNDS['max_depth']),
        random.randint(*PARAM_BOUNDS['min_samples_split'])
```

```
    ]


# ========================================
# 5. SELECTION
# ========================================

def selection(population, fitnesses, k=3):

    selected_indices = random.sample(
        range(len(population)), k
    )

    best_index = max(
        selected_indices,
        key=lambda i: fitnesses[i]
    )

    return population[best_index].copy()


# ========================================
# 6. CROSSOVER
# ========================================

def crossover(parent1, parent2, crossover_rate=0.8):

    if random.random() < crossover_rate:

        point = random.randint(1, len(parent1) - 1)

        child1 = parent1[:point] + parent2[point:]
        child2 = parent2[:point] + parent1[point:]

        return child1, child2

    return parent1.copy(), parent2.copy()


# ========================================
```

```python
# 7. MUTATION
# ==========================================

def mutate(individual, mutation_rate=0.2):

    keys = list(PARAM_BOUNDS.keys())

    for i in range(len(individual)):

        if random.random() < mutation_rate:

            bounds = PARAM_BOUNDS[keys[i]]

            individual[i] = random.randint(*bounds)

    return individual


# ==========================================
# 8. GENETIC ALGORITHM
# ==========================================

def genetic_algorithm(pop_size=15, generations=8):

    random.seed(42)
    np.random.seed(42)

    population = [
        create_individual()
        for _ in range(pop_size)
    ]

    best_overall_individual = None
    best_overall_fitness = -1

    history = []

    print("\nStarting Genetic Algorithm Optimization...\n")

    for gen in range(generations):
```

```
    # Evaluate fitness of every individual
    fitnesses = [
        evaluate_fitness(ind)
        for ind in population
    ]

    # Find best individual in current generation
    best_index = np.argmax(fitnesses)

    best_gen_fitness = fitnesses[best_index]
    best_gen_individual = population[best_index]

    # Update best overall solution
    if best_gen_fitness > best_overall_fitness:

        best_overall_fitness = best_gen_fitness
        best_overall_individual = best_gen_individual.copy()

    history.append(best_gen_fitness)

    print(
        f"Generation {gen + 1:02d}/{generations} | "
        f"Best Accuracy: {best_gen_fitness:.4f} | "
        f"Parameters: {best_gen_individual}"
    )

    # Create next generation
    new_population = []

    # Elitism: preserve the best individual
    new_population.append(
        best_gen_individual.copy()
    )

    while len(new_population) < pop_size:

        # Selection
        p1 = selection(population, fitnesses)
        p2 = selection(population, fitnesses)
```

```
            # Crossover
            c1, c2 = crossover(p1, p2)

            # Mutation
            new_population.append(
                mutate(c1)
            )

            if len(new_population) < pop_size:

                new_population.append(
                    mutate(c2)
                )

        population = new_population

    # ====================================
    # FINAL MODEL EVALUATION
    # ====================================

    n_estimators, max_depth, min_samples_split = (
        best_overall_individual
    )

    final_model = RandomForestClassifier(
        n_estimators=int(n_estimators),
        max_depth=int(max_depth),
        min_samples_split=int(min_samples_split),
        random_state=42,
        n_jobs=-1
    )

    final_model.fit(X_train, y_train)

    predictions = final_model.predict(X_test)

    test_accuracy = accuracy_score(
        y_test, predictions
    )
```

```
    print("\n========== OPTIMIZATION COMPLETED ==========")

    print("Best Cross-Validation Accuracy:",
          round(best_overall_fitness, 4))

    print("\nOptimal Hyperparameters:")

    print("Number of Trees (n_estimators):",
          best_overall_individual[0])

    print("Maximum Depth (max_depth):",
          best_overall_individual[1])

    print("Minimum Samples Split (min_samples_split):",
          best_overall_individual[2])

    print("\nFinal Test Accuracy:",
          round(test_accuracy, 4))

    # Plot optimization progress
    plt.figure(figsize=(7, 4))

    plt.plot(
        range(1, generations + 1),
        history,
        marker='o'
    )

    plt.xlabel("Generation")
    plt.ylabel("Best Cross-Validation Accuracy")
    plt.title("Genetic Algorithm Optimization Progress")
    plt.grid(True)
    plt.show()

    return best_overall_individual, best_overall_fitness, test_accuracy


# ========================================
# 9. DISPLAY SEARCH SPACE
```

```python
# ========================================

def display_search_space():

    print("\n========== HYPERPARAMETER SEARCH SPACE ==========")

    for parameter, bounds in PARAM_BOUNDS.items():

        print(
            parameter,
            " : ",
            bounds[0],
            "to",
            bounds[1]
        )


# ========================================
# 10. DISPLAY GA STEPS
# ========================================

def display_ga_steps():

    print("\n========== GENETIC ALGORITHM STEPS ==========")

    print("1. Initialization: Generate a random population.")
    print("2. Fitness: Evaluate model accuracy.")
    print("3. Selection: Choose better individuals.")
    print("4. Crossover: Combine parent chromosomes.")
    print("5. Mutation: Randomly change some genes.")
    print("6. Elitism: Preserve the best individual.")
    print("7. Repeat: Continue for multiple generations.")
    print("8. Result: Return the best hyperparameters.")


# ========================================
# 11. MAIN MENU
# ========================================

while True:
```

```
    print("\n========================================")
    print(" GENETIC ALGORITHM - ML OPTIMIZATION")
    print("========================================")

    print("1. Run Genetic Algorithm Optimization")
    print("2. Display Hyperparameter Search Space")
    print("3. Display Genetic Algorithm Steps")
    print("4. Exit")

    choice = input("\nEnter your choice (1-4): ")

    if choice == "1":

        try:

            pop_size = int(
                input("Enter Population Size (default 15): ") or 15
            )

            generations = int(
                input("Enter Number of Generations (default 8): ") or 8
            )

            if pop_size < 3 or generations < 1:

                print("Population must be at least 3 and generations at
least 1.")

            else:

                genetic_algorithm(
                    pop_size=pop_size,
                    generations=generations
                )

        except ValueError:

            print("Please enter valid integer values.")
```

```
    elif choice == "2":

        display_search_space()

    elif choice == "3":

        display_ga_steps()

    elif choice == "4":

        print("\nExiting Genetic Algorithm Program. Thank you!")
        break

    else:

        print("\nInvalid choice! Please select from 1 to 4.")
```

ASSIGN 5:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import yfinance as yf

from sklearn.preprocessing import MinMaxScaler

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense, Dropout, Input


# ========================================
# 1. GLOBAL VARIABLES
# ========================================
```

```python
df = None
prices = None
scaler = None
model = None

X_train = None
y_train = None
X_test = None
y_test = None

predictions = None
actual_prices = None
history = None

ticker = ""
train_size = 0
LOOKBACK_DAYS = 60


# ==========================================
# 2. DOWNLOAD STOCK DATA
# ==========================================

def download_stock_data():

    global df, prices, ticker

    print("\n========== DOWNLOAD STOCK DATA ==========")

    ticker = input(
        "Enter Stock Ticker (default AAPL): "
    ).strip().upper() or "AAPL"

    start_date = input(
        "Enter Start Date (default 2015-01-01): "
    ).strip() or "2015-01-01"

    end_date = input(
        "Enter End Date (default today): "
```

```
    ).strip() or (
        pd.Timestamp.today() + pd.Timedelta(days=1)
    ).strftime("%Y-%m-%d")

    try:

        print("\nDownloading stock data...")

        df = yf.download(
            ticker,
            start=start_date,
            end=end_date,
            auto_adjust=True,
            progress=False
        )

        if df.empty:

            print("No stock data found. Check the ticker and dates.")
            return

        # Handle different yfinance column formats
        close_data = df["Close"]

        if isinstance(close_data, pd.DataFrame):
            close_data = close_data.iloc[:, 0]

        prices = close_data.dropna().to_frame(
            name="Close"
        )

        if len(prices) <= LOOKBACK_DAYS + 20:

            print("Not enough historical data for prediction.")
            df = None
            prices = None
            return

        print("\nStock data downloaded successfully!")
```

```
        print("Stock:", ticker)
        print("Total Records:", len(prices))
        print("First Date:", prices.index[0])
        print("Last Date:", prices.index[-1])

        print("\nFirst Five Records:")
        print(prices.head())

    except Exception as e:

        print("\nError downloading data:", e)
        df = None
        prices = None


# ========================================
# 3. DISPLAY STOCK DATA
# ========================================

def display_stock_data():

    if prices is None:

        print("\nPlease download stock data first.")
        return

    print("\n========== STOCK DATA INFORMATION ==========")

    print("Stock:", ticker)
    print("Total Records:", len(prices))

    print("\nStatistical Information:")
    print(prices.describe())

    plt.figure(figsize=(12, 5))

    plt.plot(
        prices.index,
        prices["Close"],
        label="Closing Price"
```

```
    )

    plt.title(ticker + " Historical Stock Prices")
    plt.xlabel("Date")
    plt.ylabel("Price (USD)")
    plt.legend()
    plt.grid(True)
    plt.show()


# ==========================================
# 4. CREATE SEQUENCES
# ==========================================

def create_sequences(dataset, start_target,
                     end_target, time_steps=60):

    X = []
    y = []

    for i in range(start_target, end_target):

        X.append(
            dataset[i - time_steps:i, 0]
        )

        y.append(
            dataset[i, 0]
        )

    return np.array(X), np.array(y)


# ==========================================
# 5. BUILD AND TRAIN LSTM MODEL
# ==========================================

def train_lstm_model():

    global scaler, model
```

```
    global X_train, y_train, X_test, y_test
    global predictions, actual_prices, history
    global train_size

    if prices is None:

        print("\nPlease download stock data first.")
        return

    try:

        print("\n========== LSTM MODEL TRAINING ==========")

        epochs = int(
            input("Enter Epochs (default 20): ") or 20
        )

        if epochs < 1:

            print("Epochs must be at least 1.")
            return

        # Convert prices to NumPy array
        data = prices[["Close"]].values

        # Chronological 80-20 split
        train_size = int(len(data) * 0.8)

        # Fit scaler only on training data
        scaler = MinMaxScaler(
            feature_range=(0, 1)
        )

        scaler.fit(data[:train_size])

        scaled_data = scaler.transform(data)

        # Create training sequences
        X_train, y_train = create_sequences(
            scaled_data,
```

```
        LOOKBACK_DAYS,
        train_size,
        LOOKBACK_DAYS
    )

    # Create testing sequences
    X_test, y_test = create_sequences(
        scaled_data,
        train_size,
        len(scaled_data),
        LOOKBACK_DAYS
    )

    # Reshape data for LSTM
    X_train = X_train.reshape(
        X_train.shape[0],
        X_train.shape[1],
        1
    )

    X_test = X_test.reshape(
        X_test.shape[0],
        X_test.shape[1],
        1
    )

    print("\nTraining Records:", len(X_train))
    print("Testing Records:", len(X_test))

    # Build LSTM architecture
    model = Sequential([

        Input(shape=(LOOKBACK_DAYS, 1)),

        LSTM(
            units=50,
            return_sequences=True
        ),

        Dropout(0.2),
```

```
        LSTM(
            units=50,
            return_sequences=False
        ),

        Dropout(0.2),

        Dense(
            units=25,
            activation="relu"
        ),

        Dense(
            units=1
        )
    ])

    # Compile model
    model.compile(
        optimizer="adam",
        loss="mean_squared_error"
    )

    print("\nModel Architecture:")
    model.summary()

    # Train model
    print("\nTraining Neural Network...")

    history = model.fit(
        X_train,
        y_train,
        epochs=epochs,
        batch_size=32,
        validation_split=0.1,
        shuffle=False,
        verbose=1
    )
```

```
        print("\nModel Training Completed!")

        # Predict test data
        scaled_predictions = model.predict(
            X_test,
            verbose=0
        )

        # Convert values back to original price
        predictions = scaler.inverse_transform(
            scaled_predictions
        )

        actual_prices = scaler.inverse_transform(
            y_test.reshape(-1, 1)
        )

        print("\nModel trained successfully!")

    except Exception as e:

        print("\nTraining Error:", e)


# ========================================
# 6. PLOT ACTUAL VS PREDICTED PRICES
# ========================================

def plot_predictions():

    if predictions is None or actual_prices is None:

        print("\nPlease train the model first.")
        return

    test_dates = prices.index[train_size:]

    plt.figure(figsize=(13, 6))

    plt.plot(
```

```
        test_dates,
        actual_prices.flatten(),
        label="Actual Stock Price",
        color="black"
    )

    plt.plot(
        test_dates,
        predictions.flatten(),
        label="Predicted Stock Price",
        color="green"
    )

    plt.title(
        ticker + " Stock Price Prediction Using LSTM"
    )

    plt.xlabel("Date")
    plt.ylabel("Price (USD)")

    plt.legend()
    plt.grid(True)
    plt.show()

    # Plot training and validation loss
    plt.figure(figsize=(8, 4))

    plt.plot(
        history.history["loss"],
        label="Training Loss"
    )

    plt.plot(
        history.history["val_loss"],
        label="Validation Loss"
    )

    plt.title("LSTM Model Training Loss")
    plt.xlabel("Epoch")
    plt.ylabel("Mean Squared Error")
```

```
    plt.legend()
    plt.grid(True)
    plt.show()


# ========================================
# 7. PREDICT NEXT TRADING DAY PRICE
# ========================================

def predict_next_day():

    if model is None or scaler is None:

        print("\nPlease train the model first.")
        return

    try:

        # Take the latest 60 trading days
        last_60_days = prices[
            ["Close"]
        ].values[-LOOKBACK_DAYS:]

        # Scale the data
        scaled_last_60 = scaler.transform(
            last_60_days
        )

        # Reshape for LSTM
        X_next = scaled_last_60.reshape(
            1,
            LOOKBACK_DAYS,
            1
        )

        # Predict next price
        scaled_prediction = model.predict(
            X_next,
            verbose=0
```

```
        )

        predicted_price = scaler.inverse_transform(
            scaled_prediction
        )[0][0]

        last_price = prices["Close"].iloc[-1]

        print("\n========== NEXT DAY PREDICTION ==========")

        print(
            "Last Available Closing Price: $",
            round(last_price, 2)
        )

        print(
            "Predicted Next Trading Day Price: $",
            round(predicted_price, 2)
        )

        # Indicative price direction
        if predicted_price > last_price * 1.001:

            print("Predicted Direction: UP")

        elif predicted_price < last_price * 0.999:

            print("Predicted Direction: DOWN")

        else:

            print("Predicted Direction: NEARLY STABLE")

    except Exception as e:

        print("\nPrediction Error:", e)


# ========================================
# 8. MAIN MENU
```

```
# ==========================================

while True:

    print("\n==========================================")
    print("   STOCK MARKET PREDICTION USING LSTM")
    print("==========================================")

    print("1. Download Historical Stock Data")
    print("2. Display Stock Data and Price Graph")
    print("3. Train LSTM Neural Network")
    print("4. Display Actual vs Predicted Graph")
    print("5. Predict Next Trading Day Price")
    print("6. Exit")

    choice = input("\nEnter your choice (1-6): ")

    if choice == "1":

        download_stock_data()

    elif choice == "2":

        display_stock_data()

    elif choice == "3":

        train_lstm_model()

    elif choice == "4":

        plot_predictions()

    elif choice == "5":

        predict_next_day()

    elif choice == "6":

        print("\nExiting Stock Market Prediction Program. Thank you!")
```

```
        break

    else:

        print("\nInvalid choice! Please select from 1 to 6.")
```

ASSIGN 6:

```
import numpy as np
import matplotlib.pyplot as plt

# ====================================
# GLOBAL VARIABLES
# ====================================

city_coords = None
dist_matrix = None
pheromone = None

best_path = None
best_distance = float('inf')
history = []

n_cities = 10

n_ants = 20
n_best = 5
n_iterations = 40

alpha = 1.0
beta = 3.0
decay = 0.1
```

```python
# ====================================
# 1. GENERATE CITIES
# ====================================

def generate_cities():

    global city_coords, dist_matrix
    global pheromone, best_path, best_distance, history

    np.random.seed(42)

    city_coords = np.random.rand(n_cities, 2) * 100

    # Calculate distance matrix
    dist_matrix = np.sqrt(
        np.sum(
            (city_coords[:, np.newaxis, :] -
             city_coords[np.newaxis, :, :]) ** 2,
            axis=-1
        )
    )

    # Initialize pheromone
    pheromone = np.ones((n_cities, n_cities))

    best_path = None
    best_distance = float('inf')
    history = []

    print("\nCities generated successfully!")

    for i, (x, y) in enumerate(city_coords):
        print(f"City {i}: ({x:.2f}, {y:.2f})")


# ====================================
# 2. DISPLAY DISTANCE MATRIX
# ====================================

def display_distance():
```

```
    if dist_matrix is None:
        print("\nGenerate cities first!")
        return

    print("\nDistance Matrix:")
    print(np.round(dist_matrix, 2))


# ====================================
# 3. SELECT NEXT CITY
# ====================================

def select_next_city(current, visited):

    # Get pheromone and distance values
    p = pheromone[current].copy()
    d = dist_matrix[current].copy()

    # Ignore visited cities
    p[list(visited)] = 0

    # Calculate heuristic value
    h = np.zeros(n_cities)

    for city in range(n_cities):

        if city not in visited and d[city] > 0:
            h[city] = 1 / d[city]

    # Calculate probability
    probability = (p ** alpha) * (h ** beta)

    probability = probability / probability.sum()

    # Select next city
    return np.random.choice(n_cities, p=probability)


# ====================================
```

```python
# 4. GENERATE PATH FOR ONE ANT
# ====================================

def generate_path():

    path = []
    visited = {0}

    current = 0

    for _ in range(n_cities - 1):

        next_city = select_next_city(current, visited)

        path.append((current, next_city))

        visited.add(next_city)
        current = next_city

    # Return to starting city
    path.append((current, 0))

    return path


# ====================================
# 5. CALCULATE PATH DISTANCE
# ====================================

def calculate_distance(path):

    distance = 0

    for start, end in path:
        distance += dist_matrix[start][end]

    return distance


# ====================================
```

```
# 6. RUN ACO ALGORITHM
# ====================================

def run_aco():

    global pheromone, best_path, best_distance, history

    if dist_matrix is None:
        print("\nGenerate cities first!")
        return

    best_path = None
    best_distance = float('inf')
    history = []

    pheromone = np.ones((n_cities, n_cities))

    for iteration in range(n_iterations):

        all_paths = []

        # Generate paths for all ants
        for ant in range(n_ants):

            path = generate_path()

            distance = calculate_distance(path)

            all_paths.append((path, distance))

        # Find best path in current iteration
        current_path, current_distance = min(
            all_paths, key=lambda x: x[1]
        )

        # Update overall best path
        if current_distance < best_distance:

            best_distance = current_distance
            best_path = current_path
```

```
        # Pheromone evaporation
        pheromone *= (1 - decay)

        # Sort paths by distance
        all_paths.sort(key=lambda x: x[1])

        # Deposit pheromone on best paths
        for path, distance in all_paths[:n_best]:

            for start, end in path:

                pheromone[start][end] += 1 / distance
                pheromone[end][start] += 1 / distance

        history.append(best_distance)

        print(
            f"Iteration {iteration + 1}: "
            f"Best Distance = {best_distance:.2f}"
        )

    print("\nACO Completed Successfully!")
    print("Minimum Distance:", round(best_distance, 2))


# ====================================
# 7. DISPLAY BEST ROUTE
# ====================================

def display_route():

    if best_path is None:
        print("\nRun ACO first!")
        return

    ordered_cities = [0]

    for start, end in best_path:
        ordered_cities.append(end)
```

```
    print("\nBest Route:")

    print(" -> ".join(
        f"City {city}" for city in ordered_cities
    ))

    print("Minimum Distance:", round(best_distance, 2))

    # Plot route
    plt.figure(figsize=(8, 5))

    plt.plot(
        city_coords[ordered_cities, 0],
        city_coords[ordered_cities, 1],
        'o-'
    )

    for i, (x, y) in enumerate(city_coords):
        plt.annotate(f"City {i}", (x, y))

    plt.title("Best Route using ACO")
    plt.xlabel("X Coordinate")
    plt.ylabel("Y Coordinate")
    plt.grid()
    plt.show()


# ====================================
# 8. DISPLAY CONVERGENCE GRAPH
# ====================================

def display_convergence():

    if not history:
        print("\nRun ACO first!")
        return

    plt.plot(range(1, len(history) + 1), history)
```

```
    plt.title("ACO Convergence Graph")
    plt.xlabel("Iteration")
    plt.ylabel("Best Distance")
    plt.grid()
    plt.show()


# ====================================
# 9. MAIN MENU
# ====================================

while True:

    print("\n================================")
    print(" TRAVELLING SALESMAN USING ACO")
    print("================================")

    print("1. Generate Cities")
    print("2. Display Distance Matrix")
    print("3. Run ACO")
    print("4. Display Best Route")
    print("5. Display Convergence Graph")
    print("6. Exit")

    choice = input("\nEnter your choice: ")

    if choice == "1":
        generate_cities()

    elif choice == "2":
        display_distance()

    elif choice == "3":
        run_aco()

    elif choice == "4":
        display_route()

    elif choice == "5":
        display_convergence()
```

```
    elif choice == "6":
        print("Exiting Program!")
        break

    else:
        print("Invalid choice!")
```

ASSIGN 7:

```
import numpy as np
import matplotlib.pyplot as plt

from sklearn.datasets import make_blobs
from sklearn.cluster import KMeans


# ========================================
# GLOBAL VARIABLES
# ========================================

X = None

pso_centroids = None
pso_labels = None
pso_sse = None

kmeans = None
kmeans_labels = None

history = []

N_CLUSTERS = 4
N_PARTICLES = 30
MAX_ITERATIONS = 80
```

```
w = 0.6
c1 = 1.8
c2 = 1.8


# ========================================
# 1. GENERATE DATASET
# ========================================

def generate_dataset():

    global X
    global pso_centroids, pso_labels, pso_sse
    global kmeans, kmeans_labels, history

    # Generate sample data
    X, y = make_blobs(
        n_samples=400,
        centers=N_CLUSTERS,
        cluster_std=0.8,
        random_state=42
    )

    # Reset previous results
    pso_centroids = None
    pso_labels = None
    pso_sse = None
    kmeans = None
    kmeans_labels = None
    history = []

    print("\nDataset generated successfully!")
    print("Total data points:", len(X))
    print("Number of clusters:", N_CLUSTERS)

    print("\nFirst five data points:")
    print(X[:5])
```

```
# ========================================
# 2. DISPLAY DATASET
# ========================================

def display_dataset():

    if X is None:
        print("\nGenerate dataset first!")
        return

    plt.figure(figsize=(7, 5))

    plt.scatter(X[:, 0], X[:, 1])

    plt.title("Original Dataset")
    plt.xlabel("Feature 1")
    plt.ylabel("Feature 2")

    plt.grid()
    plt.show()


# ========================================
# 3. FITNESS FUNCTION
# ========================================

def calculate_fitness(data, centroids):

    # Calculate distance of each point from each centroid
    distances = np.linalg.norm(
        data[:, np.newaxis, :] - centroids[np.newaxis, :, :],
        axis=2
    )

    # Find minimum distance for every data point
    min_distances = np.min(distances, axis=1)

    # Calculate Sum of Squared Errors
    sse = np.sum(min_distances ** 2)
```

```
    return sse


# ========================================
# 4. ASSIGN DATA POINTS TO CLUSTERS
# ========================================

def assign_clusters(data, centroids):

    distances = np.linalg.norm(
        data[:, np.newaxis, :] - centroids[np.newaxis, :, :],
        axis=2
    )

    labels = np.argmin(distances, axis=1)

    return labels


# ========================================
# 5. RUN PARTICLE SWARM OPTIMIZATION
# ========================================

def run_pso():

    global pso_centroids, pso_labels, pso_sse, history

    if X is None:
        print("\nGenerate dataset first!")
        return

    np.random.seed(42)

    n_samples, dimensions = X.shape

    data_min = np.min(X, axis=0)
    data_max = np.max(X, axis=0)

    # Initialize particle positions
    positions = np.random.uniform(
```

```
        data_min,
        data_max,
        size=(N_PARTICLES, N_CLUSTERS, dimensions)
    )

    # Initialize velocities
    velocities = np.zeros_like(positions)

    # Initialize personal best positions
    pbest_positions = positions.copy()

    pbest_scores = np.full(N_PARTICLES, float('inf'))

    # Initialize global best
    gbest_position = None
    gbest_score = float('inf')

    history = []

    print("\n========== RUNNING PSO ==========")

    for iteration in range(MAX_ITERATIONS):

        # Evaluate fitness of every particle
        for i in range(N_PARTICLES):

            score = calculate_fitness(
                X,
                positions[i]
            )

            # Update personal best
            if score < pbest_scores[i]:

                pbest_scores[i] = score
                pbest_positions[i] = positions[i].copy()

            # Update global best
            if score < gbest_score:
```

```
            gbest_score = score
            gbest_position = positions[i].copy()

    # Update velocity and position
    for i in range(N_PARTICLES):

        r1 = np.random.rand(N_CLUSTERS, dimensions)
        r2 = np.random.rand(N_CLUSTERS, dimensions)

        # Cognitive component
        cognitive = c1 * r1 * (
            pbest_positions[i] - positions[i]
        )

        # Social component
        social = c2 * r2 * (
            gbest_position - positions[i]
        )

        # Velocity update
        velocities[i] = (
            w * velocities[i]
            + cognitive
            + social
        )

        # Position update
        positions[i] += velocities[i]

        # Keep centroids within data boundaries
        positions[i] = np.clip(
            positions[i],
            data_min,
            data_max
        )

    history.append(gbest_score)

    print(
        f"Iteration {iteration + 1}/{MAX_ITERATIONS} "
```

```
            f"| Best SSE: {gbest_score:.2f}"
        )

    # Store final results
    pso_centroids = gbest_position
    pso_sse = gbest_score

    pso_labels = assign_clusters(
        X,
        pso_centroids
    )

    print("\nPSO Clustering Completed!")
    print("Final PSO SSE:", round(pso_sse, 2))


# ========================================
# 6. DISPLAY PSO CLUSTERING GRAPH
# ========================================

def display_pso_result():

    if pso_centroids is None:
        print("\nRun PSO first!")
        return

    plt.figure(figsize=(7, 5))

    plt.scatter(
        X[:, 0],
        X[:, 1],
        c=pso_labels,
        cmap='viridis',
        alpha=0.6
    )

    plt.scatter(
        pso_centroids[:, 0],
        pso_centroids[:, 1],
        color='red',
```

```
        marker='X',
        s=200,
        label='PSO Centroids'
    )

    plt.title(f"PSO Clustering (SSE: {pso_sse:.2f})")

    plt.xlabel("Feature 1")
    plt.ylabel("Feature 2")

    plt.legend()
    plt.grid()
    plt.show()


# ========================================
# 7. COMPARE WITH K-MEANS
# ========================================

def compare_kmeans():

    global kmeans, kmeans_labels

    if X is None:
        print("\nGenerate dataset first!")
        return

    if pso_centroids is None:
        print("\nRun PSO first!")
        return

    # Apply K-Means
    kmeans = KMeans(
        n_clusters=N_CLUSTERS,
        random_state=42,
        n_init=10
    )

    kmeans.fit(X)
```

```
kmeans_labels = kmeans.labels_

print("\n========== CLUSTERING COMPARISON ==========")

print("PSO SSE:", round(pso_sse, 2))

print("K-Means SSE:", round(kmeans.inertia_, 2))

# Display both results
fig, axes = plt.subplots(1, 2, figsize=(12, 5))

# PSO graph
axes[0].scatter(
    X[:, 0],
    X[:, 1],
    c=pso_labels,
    cmap='viridis',
    alpha=0.6
)

axes[0].scatter(
    pso_centroids[:, 0],
    pso_centroids[:, 1],
    color='red',
    marker='X',
    s=180,
    label='Centroids'
)

axes[0].set_title("PSO Clustering")
axes[0].legend()

# K-Means graph
axes[1].scatter(
    X[:, 0],
    X[:, 1],
    c=kmeans_labels,
    cmap='viridis',
    alpha=0.6
)
```

```
    axes[1].scatter(
        kmeans.cluster_centers_[:, 0],
        kmeans.cluster_centers_[:, 1],
        color='red',
        marker='X',
        s=180,
        label='Centroids'
    )

    axes[1].set_title("K-Means Clustering")
    axes[1].legend()

    plt.tight_layout()
    plt.show()


# ========================================
# 8. DISPLAY CONVERGENCE GRAPH
# ========================================

def display_convergence():

    if not history:
        print("\nRun PSO first!")
        return

    plt.figure(figsize=(8, 5))

    plt.plot(
        range(1, len(history) + 1),
        history
    )

    plt.title("PSO Convergence Graph")
    plt.xlabel("Iteration")
    plt.ylabel("Best SSE")

    plt.grid()
    plt.show()
```

```
# ========================================
# 9. MAIN MENU
# ========================================

while True:

    print("\n========================================")
    print(" DATA CLUSTERING USING PSO")
    print("========================================")

    print("1. Generate Dataset")
    print("2. Display Dataset")
    print("3. Run PSO Clustering")
    print("4. Display PSO Clustering Result")
    print("5. Compare PSO with K-Means")
    print("6. Display Convergence Graph")
    print("7. Exit")

    choice = input("\nEnter your choice (1-7): ")

    if choice == "1":
        generate_dataset()

    elif choice == "2":
        display_dataset()

    elif choice == "3":
        run_pso()

    elif choice == "4":
        display_pso_result()

    elif choice == "5":
        compare_kmeans()

    elif choice == "6":
        display_convergence()
```

```
    elif choice == "7":
        print("\nExiting Program. Thank You!")
        break

    else:
        print("\nInvalid choice! Enter a number from 1 to 7.")
```